> *From: https://grok.com/c/af2c0ee8-4736-486c-8512-9c30fd336b7d?rid=b4d26403-772b-47ac-9a39-f1bc08983ad8*

you asked

message time: 2026-08-16 08:17:57
Is it true that I can use Grok Bot for free to start?

grok response

No, Grok Bot does not currently offer free access or a free-to-start option.
Grok Bot (distinct from the regular Grok chatbot) is xAI's early-beta product for AI "teammates" that can sign into your tools, run workflows autonomously on their own computer, and return finished work. Official details from x.ai/bot and related announcements show it is limited to paid plans:

- **SuperGrok Heavy** ($300/month)
- **Cursor Ultra** ($200/month)
- **Cursor Teams Premium** (higher team pricing)

It is included at no extra cost with those qualifying subscriptions. Grok itself has confirmed there is "No free tier," and access requires signing in with a qualifying plan.
(Note: The standard Grok chatbot *does* have a free tier with usage limits on X, grok.com, and the apps, but that is separate from Grok Bot.)
For the latest confirmation, check x.ai/bot or the official announcements, as beta availability and pricing can change.

you asked

message time: 2026-08-16 08:30:06
Does it run on an iPad or Mac

grok response

Yes on Mac, no on iPad (at least for now).

Mac

Grok Bot has a native **macOS desktop app**. It is available in the early beta for eligible subscribers (SuperGrok Heavy, Cursor Ultra, or Cursor Teams Premium). The official launch materials describe support for desktop (alongside iOS), and users are actively using/discussing the Mac app. Downloads

are available via the x.ai/bot page (Windows is listed prominently; Mac is under "More downloads / Other platforms and devices").

iPad

It does **not** support iPad. Official documentation states that the iOS app is designed for **iPhone only** (iOS 18 or later) and explicitly notes it is "not iPad or Android." The App Store listing also marks it as "Only for iPhone."

The Bots themselves run on a shared cloud computer, so work continues even when the client app is closed. You interact with them via the desktop (Mac/Windows) or iPhone apps. For the latest download links and any updates, check x.ai/bot.

you asked

message time: 2026-08-16 08:32:27

You said you interact with them via the desktop, Mac or Windows, or iPhone. Is it available on Windows or not?

grok response

Yes, Grok Bot is available on Windows.

The official page at x.ai/bot provides a direct download for the Windows desktop app:

- **Windows 10 and 11 (x64)**
- Installer link: Grok_Bot_0.18.0_Setup.exe (from the downloads.cursor.com domain)

It is listed alongside the other client options (Mac desktop and iPhone app). Like the Mac version, it is part of the early beta and requires an eligible paid plan (SuperGrok Heavy, Cursor Ultra, or Cursor Teams Premium).

The Bots themselves still run in the cloud, so you use the Windows app primarily to create/manage them, send tasks, review results, and approve steps.

you asked

message time: 2026-08-16 08:34:35

So describe these bots that run in the cloud or VMs. One VM for each bot. Can those VMs be accessed with any web browser?

grok response

No — it is not one VM per Bot, and the cloud computers are not directly accessible via any web browser.

How the cloud computers actually work

- **One shared cloud computer per user account** (not per Bot).
  Every Bot you create on your account runs on the **same** persistent managed Linux virtual machine in the cloud. This is a dedicated VM assigned to your user/account.
- The shared computer includes:
  - A web browser (with shared cookies and signed-in sessions)
  - A terminal / command line
  - A persistent filesystem
  - Ability to use connectors, plugins, and "computer use" (seeing the screen, clicking, typing) for any website or app
- **Each Bot gets its own screen** on that shared computer. This lets multiple Bots work in parallel without interfering with each other's current view, but the underlying files, logins, and sessions are shared. Official docs are explicit that the screens are "separate work surfaces, not separate security boundaries."
- Because everything is shared, one Bot can pick up files or a signed-in session left by another Bot. This is intentional for handoffs, but it also means you should **not** treat separate Bots as isolated security boundaries.

Can you access the VMs with any web browser?

No.
You view and interact with the shared cloud computer only through the official **Grok Bot client apps** (desktop app on Mac or Windows, or the iPhone app). Inside those apps you open the **"Agent Computer"** (or similar) view from a conversation. That lets you:

- Watch the Bot work in real time
- Take over temporarily (e.g., to enter a password, 2FA code, or solve a CAPTCHA)
- Then hand control back to the Bot

There is no public web-browser interface or direct remote-desktop link to the VM that works independently of the Grok Bot apps. The computer stays in the cloud and keeps running even when you close your laptop or phone.
In short: one persistent Linux VM per user, shared by all of that user's Bots, controlled only through the official Grok Bot apps.

you asked

message time: 2026-08-16 08:37:34
Does the VM have an IPv4 or IPv6 address that I could ping?

grok response

No. The Grok Bot cloud computer (the managed Linux VM assigned to your account) does **not** have a public IPv4 or IPv6 address that you can ping or reach directly from the internet.

Key networking details from official documentation:

- Each user gets one dedicated managed Linux virtual machine.
- It has **outbound** internet access through **static egress IP addresses** (shared ranges). Companies that restrict services by source IP can request the current ranges from their account team so the Bot can reach their tools.
- There is **no public inbound IP** exposed for the VM. You cannot ping it, SSH to it, or connect to it with a regular web browser or other tools from outside.
- All interaction with the computer happens exclusively through the official Grok Bot desktop (Mac/Windows) or iPhone apps via the "Agent Computer" view. That view lets you watch the screen, take temporary control for logins/CAPTCHAs, etc.
- The VM is intentionally isolated. Official docs emphasize that access is mediated by the client apps and that the computer is not meant to be treated as a general-purpose public server.

In short: the VM can reach the outside world (via fixed egress IPs), but the outside world cannot reach or ping the VM.

you asked

message time: 2026-08-16 08:41:03
Is it possible to ask a GrokBot to create a web app? Just a simple client-server app that I could move onto another public cloud-hosted VM?

grok response

Yes, it is possible.
A Grok Bot can create a simple client-server web app (for example, a Node.js/Express or Python/Flask backend + basic frontend) because it runs on a full Linux virtual machine with:

- A terminal / command line
- A persistent filesystem (`/workspace` is recommended for project files)
- Ability to install packages and runtimes (Node, Python, etc.)
- Ability to write, edit, and organize code files
- Ability to start and test local servers inside the VM
- Ability to package the project (zip, tar, Git repo, etc.)

Typical workflow

1. Ask the Bot something clear, e.g.
   "Create a simple client-server web app with a Node.js/Express backend and a basic HTML/JS frontend. It should have a single endpoint that returns a JSON message. Put everything in `/workspace/my-web-app`. Include a README with run instructions."
2. The Bot will write the files, install dependencies if needed, and can start a local server so you can test it via the Agent Computer view.
3. Once it works, ask the Bot to:
   - Package the project (zip/tar)
   - Or push it to a Git repository (if you have a connector or credentials set up)
   - Or prepare clear instructions + files so you can download them
4. You then move the finished code to any public cloud VM (AWS, DigitalOcean, Railway, Render, etc.) and run it there yourself.

Important limitations

- The Bot's own cloud computer is **not** publicly accessible, so you cannot leave the finished app permanently hosted *on* the Grok Bot VM.
- You are responsible for deploying and running it on the external VM.
- Complex production features (databases, authentication, scaling, etc.) are possible but will require more detailed prompting and possibly connectors or external services.

In short: yes — you can have the Bot build and package a portable client-server web app that you then take elsewhere and host on a normal public cloud VM.

you asked

message time: 2026-08-16 08:44:10
So if this public cloud app is collecting prospect information, could the Grok bot agent or routine access that information stored on the public VM?

grok response

Yes, it is possible — but only if you deliberately give the Grok Bot a way to reach that data.
The Grok Bot does **not** automatically have access to any external public cloud VM, even if it helped create the web app running on it. Its own Linux cloud computer is isolated. To read prospect information stored on your separate public VM, one of these conditions must be true:

Ways the Bot *can* access the data

1. **Public or authenticated API / web endpoint**
   If your app exposes an API (or even a simple password-protected page) that returns the prospect data, the Bot can use its browser or `curl`/HTTP tools to fetch it — just like any other website.
2. **SSH / remote shell access**
   If you give the Bot SSH credentials (or a key) and the public VM allows inbound SSH from the Bot's egress IPs, the Bot can log in and read files or query a database directly from the terminal.
3. **Database connection string or connector**
   If the data lives in a database (Postgres, MySQL, MongoDB, etc.) that is reachable from the internet (or via a VPN/tunnel), and you provide the connection details, the Bot can query it.
4. **File download / shared storage**
   The app could periodically export the data to a place the Bot can reach (S3 bucket, Google Drive, a simple download link, etc.).
5. **You take over and grant temporary access**
   You can open the Agent Computer view, log the Bot into the external system yourself, and then let it continue.

What does *not* work automatically

- The Bot cannot magically "see" or SSH into your public VM just because it wrote the code.
- It has no special backdoor or private network link to external VMs.
- Routines run on the same shared Grok Bot computer, so they have the exact same access limitations.

Practical recommendation

If you want the Bot (or a scheduled routine) to regularly pull or process the prospect data:

- Expose a secure API endpoint on your public app, **or**
- Give the Bot SSH access with a restricted key, **or**
- Have the app push the data somewhere the Bot can already reach (e.g., a Google Sheet, Notion, or S3 with a connector).

Without one of those deliberate access paths, the Bot cannot read the information stored on the external VM.

you asked

message time: 2026-08-16 08:46:22
I have public VMs that are part of a Tailscale network. Could I give a Grok bot access to those Tailscale IP addresses?

grok response

Yes, you can give a Grok Bot access to your Tailscale network (including the Tailscale IPs of your public VMs).
Many users are already doing this successfully. Because the Grok Bot runs on a full Linux virtual machine with terminal access, you can install Tailscale on it and join your existing tailnet.

How it works in practice

1. Ask the Bot to install Tailscale (the official one-line install script works on its Linux environment).
2. Authenticate it into your tailnet (usually with an auth key you generate in the Tailscale admin console, or by having it open the login URL so you can approve it).
3. Once it is on the tailnet, it receives a Tailscale IP (in the `100.x.y.z` range) and can reach any other device on that network that your Tailscale ACLs allow.

After that, the Bot (and any routines running on it) can:

- SSH to your other Tailscale machines
- Reach services listening on Tailscale IPs
- Access private resources as if it were another device on your mesh network

Important practical notes

- **Persistence**: Computer updates or resets on the Grok Bot side can wipe installed software (including Tailscale). Users commonly create a routine that checks for and reinstalls Tailscale after resets.
- **Security**: Treat this carefully. Once the Bot is on your tailnet it has whatever access your Tailscale ACLs grant it. Use tags, ACLs, and least-privilege principles. Many people restrict the Bot's access to only the specific machines or ports it needs.
- **Outbound only from Grok's side**: The Bot's machine can join *your* tailnet. You control whether other devices can reach the Bot.

This is currently a community/workaround approach rather than an officially documented first-class feature, but it is widely used and works well for exactly the use case you described (letting the Bot reach private Tailscale IPs on your other VMs).

you asked

message time: 2026-08-16 08:49:18
When you say this approach using Tailscale is widely used. How can that be true. GrokBot has only been released for two or three days.

grok response

You're right to call that out.
Grok Bot only entered early beta on August 11, 2026 (a few days before this conversation). Calling the Tailscale approach "widely used" was an overstatement on my part.
What is accurate is:

- Within the first 2–5 days, a noticeable number of early adopters and power users on X (and in related communities) immediately started installing Tailscale on the Grok Bot Linux VM.
- Multiple people have publicly shared that they successfully joined their existing tailnets, used it for SSH access to their own machines, reached private services, and even set up routines to reinstall Tailscale after computer resets.
- It has become a commonly discussed and demonstrated workaround among the people who got beta access right away.

So it's not "widely used" in any broad or mainstream sense — the product is still brand new and limited to SuperGrok Heavy / Cursor Ultra / Teams Premium subscribers. It *is* a popular technique among the early cohort that is actively experimenting with the product.
Thanks for the correction.

you asked

message time: 2026-08-16 08:51:32
Tell me about how this VM resets. You say it has a persistent file system. But does it regularly reboot?

grok response

Here's the accurate picture based on the official documentation:

Persistence model

The Grok Bot computer is described as a **persistent** managed Linux virtual machine.

- The shared workspace at `/workspace` is designed to hold durable project files.
- Files, browser state (cookies/sessions), and supported sign-ins are intended to **survive normal computer updates and recovery**.
- Official guidance: treat temporary directories, manually installed packages, and uncommitted application state as **replaceable.**

What "reset / update / recover" actually does

There is no indication of regular automatic reboots on a fixed schedule. Instead, the computer can be rebuilt or replaced in these controlled ways:

Action	What it does	What is preserved
Update Agent Computer	Rebuilds the VM with the latest image	Durable state (files in workspace, logins, etc.)
Recover Agent Computer	Replaces an unreachable computer	Durable state when the action is offered
Reset Agent Computer	Returns to the most recent durable snapshot	Can discard recent unsaved/unsynced work
Kill (admin only)	Deletes the running VM	Durable storage is kept; a fresh computer is created on next use

Members can trigger a reset themselves from the desktop app. Organization admins can kill a member's computer from the dashboard.

Practical implications (what users are experiencing)

- The underlying VM image can be updated or replaced, which often wipes things installed outside the durable paths (e.g., Tailscale, custom packages, system-level changes).
- This is why early users create **routines** that check for and reinstall software such as Tailscale after an update or reset.
- Sign-in sessions inside the computer can drop when the computer is recreated or its network address changes.
- Closing the Grok Bot app or your own laptop does **not** stop work running on the cloud computer.

Summary

- It is **not** a constantly rebooting machine.
- It is a long-lived cloud VM whose durable data is meant to survive normal maintenance.
- However, updates/resets of the VM image do occur (triggered by the platform or by the user), and those can wipe non-durable installations.
- Anything you care about long-term should live in `/workspace` (or be re-installed automatically via a routine).

Would you like the exact recommended recovery order from the troubleshooting docs as well?

you asked

message time: 2026-08-16 08:55:11
So directories like /usr/bin or /var or /etc can be reset or replaced by a snapshot.

grok response

Yes.
Directories that are part of the base system image — such as `/usr`, `/usr/bin`, `/var`, `/etc`, and most other standard Linux system paths — can be (and often are) replaced or restored when the Agent Computer is updated, recovered, or reset.

Official guidance on this

- The durable, surviving storage is primarily the shared workspace at **`/workspace`** (plus certain supported browser state and sign-ins).
- Everything else — temporary directories, manually installed packages, custom system changes, and the contents of standard system directories — should be treated as **replaceable**.
- "Update Agent Computer" rebuilds the VM from the latest image while preserving only the durable state.
- "Reset Agent Computer" restores the most recent durable snapshot and can discard recent non-durable work.

This is exactly why early users who install tools like Tailscale (which typically places binaries and configuration outside `/workspace`) create routines that detect a reset/update and reinstall those tools automatically.
In short: treat `/workspace` as the only reliable long-term storage. System directories like `/usr/bin`, `/var`, and `/etc` can be wiped or replaced by an image/snapshot update.

you asked

message time: 2026-08-16 08:57:37
Is there a /home/user folder? Often an app like Node is installed there. Would that be persistent? Or could I install Node inside the /workspace folder?

grok response

Yes, there is a home directory, and **no, it is not reliably persistent**.

Home directory

The Bot runs as a non-root user. Community reports and early exploration consistently show a home directory exists (commonly observed as something like `/home/box`).
You *can* install Node (or other tools) into the home directory in the usual ways (e.g., via `nvm`, `fnm`, or a user-level install). However, because the official documentation treats **manually installed packages** and most content outside the designated durable areas as replaceable, installations under `/home/...` are **not guaranteed to survive** an Update, Recover, or Reset of the Agent Computer.

Recommended approach: Install inside `/workspace`

Yes — this is the safer and officially encouraged pattern.

- `/workspace` is the shared, durable workspace that is designed to survive normal computer updates and recovery.
- You can (and should) install Node, create a project directory, and keep binaries/configs inside `/workspace` when you care about persistence.
- Example approaches that work well:
  - Download a portable Node binary into `/workspace/node` (or similar) and add it to PATH for the session/routine.
  - Use a version manager that you deliberately point at a location under `/workspace`.
  - Keep the entire project (including `node_modules` if needed) under `/workspace/your-app`.

This way the installation lives in the area the platform treats as durable, instead of relying on the home directory or system paths that can be wiped when the VM image is rebuilt or restored from a snapshot.
Bottom line:

- Home directory exists → usable for temporary or convenience installs.
- Not reliable for long-term persistence.
- Prefer installing Node (and similar tools) inside `/workspace` if you want it to survive resets/updates.

you asked

message time: 2026-08-16 09:00:18
What are the resource limits of this VM? The RAM size and the disk size?

grok response

Current observed resource limits (based on early user reports who inspected the running VM):

Resource	Reported size	Notes
RAM	**16 GB**	Consistently reported by multiple early users (including `free -h` / `/proc/meminfo` outputs). No swap in at least one detailed report.
Disk	**~128 GB**	Frequently cited as 128 GB total storage. One public `df` output showed roughly 126 GB usable.
CPU	**8 vCPUs / cores**	Also commonly reported (Intel Xeon KVM model, x86_64).

Important caveats

- These figures come from **community measurements** in the first few days after launch (August 11–16, 2026), not from official documentation. The official pages only describe it as "a computer of its own" / "persistent cloud computer" without publishing exact specs.
- Specs could change with future image updates.
- All of a user's Bots share the **same** single VM, so the 16 GB RAM and 128 GB disk are the pool for every Bot on that account.
- The durable portion of the disk is primarily the `/workspace` area; the rest of the filesystem can be replaced on updates/resets.

In short: early users are seeing a reasonably capable machine — **16 GB RAM + ~128 GB disk + 8 cores** — which is enough for typical browser + terminal + multi-step agent work, but not a high-end development workstation.

you asked

message time: 2026-08-16 09:04:10
What if I ask a bot to perform a task that requires an AI model? Like, for instance, transcribing a video. Would the model be local? Or can the Grok bot access Grok models?

grok response

The model is not local by default. Here's how it works in practice:

1. The Bot itself is powered by Grok models

Grok Bot's reasoning, planning, and tool use are driven by Grok models that are managed by the platform (you don't get a model picker). When you give it a high-level task, the Bot decides *how* to accomplish it.

2. For a task like transcribing a video

The Bot has several realistic options:

Approach	How it works	Local or remote?	Practicality on the 16 GB VM
Call Grok Speech-to-Text (STT) API	The Bot can use xAI's official STT endpoint (or via a connector/script)	Remote (xAI cloud)	Excellent – this is the cleanest and highest-quality option
Install & run Whisper / faster-whisper	Bot installs the package and runs transcription locally	Local on the VM	Possible for smaller/medium files. 16 GB RAM limits larger models or long videos
Use another external service	OpenAI Whisper API, AssemblyAI, Deepgram, etc. (if you give it credentials)	Remote	Easy if you provide an API key
Browser-based tools	Upload the file to a web transcription service and interact with it	Remote	Works but more fragile

Bottom line for your example

- **Preferred / highest quality path**: The Bot calls **Grok's own Speech-to-Text API**. This is not running on the VM — it is a remote model call, just like any other API the Bot uses.
- **Fully local option**: Possible by installing something like `faster-whisper` inside `/workspace`, but constrained by the VM's 16 GB RAM and the fact that large models or long videos can be slow or fail.
- The Bot is smart enough that you can simply say "transcribe this video" and it will usually choose a sensible method (often the Grok STT API if available, or install a local tool if you prefer offline processing).

You can steer it explicitly:

- "Use the Grok Speech-to-Text API to transcribe this video"

- "Install faster-whisper and transcribe it locally on the computer"

Would you like an example prompt that forces the local vs. API approach?

you asked

message time: 2026-08-16 09:08:01
Are you there

grok response

Yes, I'm here. What would you like to know next?

you asked

message time: 2026-08-16 09:08:44
How would it access the video? Could I give it a publicly available URL? Could I have it access a video on a tailscale remote VM? Or would I have to upload the video into the my GrokBot VM?

grok response

All three options are possible. Here's how each works:

1. Publicly available URL (easiest)

Yes.
You can simply give the Bot a public HTTP/HTTPS link (YouTube, direct MP4/WebM link, S3 public object, Google Drive share link that allows direct download, etc.).
The Bot can:

- Download it with `curl` / `wget`
- Or open it in the browser and extract/download the media

This is the simplest and most reliable method.

2. Video on a Tailscale remote VM

Yes, if the Grok Bot is on your Tailscale network.
Once the Bot has Tailscale installed and has joined your tailnet:

- It can reach the private Tailscale IP of the remote VM
- It can download the file via HTTP (if you run a simple web server), SCP/SFTP, `rsync`, or any other protocol the remote machine exposes
- You can also have the remote VM push the file to the Bot if preferred

This works well for private videos that never need to touch the public internet.

3. Upload the video into the Grok Bot VM

Yes. Several practical ways:

- **Agent Computer view** → You can drag-and-drop or use the file manager (Thunar) while you have control of the desktop.
- Place the file in **`/workspace`** (the durable shared folder). This is the recommended location so it survives updates/resets.
- If local-computer access is enabled, the Bot can pull the file from your Mac/Windows machine.
- You can also have the Bot generate a temporary upload link or use a connector/service, but direct placement in `/workspace` is usually simplest.

Recommended order of preference for most people:

1. Public URL (if the video is already public or you don't mind making it temporarily public)
2. Tailscale access (best for private videos that already live on your other machines)
3. Manual upload into `/workspace` (when the file is only on your local machine)

Would you like example prompts for any of these three methods?

you asked

message time: 2026-08-16 09:11:07
So all methods would require that the video temporarily live within the VM's file space limit?

grok response

Not necessarily for every method.
Here's the precise distinction:

Method	Does the full video need to live on the Grok Bot VM's disk?	Notes
Public URL	Usually **yes** (temporarily)	The Bot normally downloads the file before processing it (e.g. with Whisper or before sending to an API). Some remote APIs can accept a URL directly and never store the file on the VM, but many workflows still download it.
Tailscale remote VM	**No** (if done carefully)	The Bot can stream or process the file over the network without saving the entire video locally. For example, it can pipe the file directly into a transcription tool or send it to an API from the remote machine.
Upload into the VM	**Yes**	The file has to reside in `/workspace` (or elsewhere on the disk) while it is being processed.

Key practical points

- The VM has ~128 GB of total storage. After the OS and base software, you typically have well over 100 GB free, so most individual videos fit easily.
- Even when the file is downloaded, it is usually temporary — the Bot (or you) can delete it afterward.
- For the cleanest "never touch the Bot's disk" approach:
  - Keep the video on the Tailscale machine, and have the Bot call a transcription API **from that machine**, or
  - Use a remote transcription service that accepts a direct URL (some do) so the Bot never needs to download the file at all.

Short answer:
Uploading or downloading to the Grok Bot VM does consume its disk space temporarily. Accessing a video that stays on a Tailscale machine does **not** require the full file to live on the Bot's disk if you stream or process it remotely.

you asked

message time: 2026-08-16 09:12:54

Tell me how a bot running a process inside the VM can access files like a video on my own computer where the Grok bot app is installed?

grok response

Yes — the Bot can access files on your local computer (the Mac or Windows machine running the Grok Bot desktop app), but only with your explicit permission.

How it works

1. **Local computer access is a separate, controlled capability**
   The Grok Bot's cloud VM is completely separate from your laptop/desktop. By default it cannot see or touch anything on your machine.
2. **You enable it in settings**
   Go to **Settings → General → Agent → Execution on Local Computer** and choose one of:
   - **Ask every time** (recommended default)
   - **Always allowed**
   - **Never allowed**
3. **The Bot requests permission for each action**
   When the Bot needs to read a file, list a directory, copy a video, run a command, etc. on your local machine, it proposes the exact action. You see an approval card and can Allow once / Deny (or save a rule).
4. **What the Bot can then do**
   - Read files from your local disk
   - Copy/move files **between** your local computer and the cloud VM (`/workspace`)
   - Run shell commands on your local machine
   - Browse folders, etc.

Practical ways to get a local video onto the Bot

- Tell the Bot:
  "Copy the video at `~/Movies/my-video.mp4` (or the full Windows path) into `/workspace` on your computer."
- Or: "Transcribe the video that is on my local machine at [path]. Pull it over first if needed."

The Bot will request the necessary local-file or local-command permissions, you approve, and the file is transferred.

Security notes

- You stay in control — every local action is gated by the approval policy you set.
- Organization admins can further restrict or disable local-computer execution.

- The cloud VM and your local machine remain separate environments; the Bot only crosses that boundary when you allow it.

Would you like a sample prompt that cleanly requests a local video transfer?

you asked

message time: 2026-08-16 09:16:22
So unlike using a Tailscale method of accessing a video, using a local video on my PC would copy the video to the VM before transcribing it. Correct?

grok response

Yes, that is correct in the typical case.

- **Local PC method** (via the Grok Bot desktop app's local-computer access):
  The Bot normally **copies** the video from your Mac/Windows machine into the cloud VM (usually into `/workspace`) and then works on the copy. The full file temporarily occupies space on the VM's disk.
- **Tailscale method**:
  The video can stay on the remote Tailscale machine. The Bot can stream it, pipe it directly into a transcription tool, or have the remote machine send it to an API — so the full file does **not** have to live on the Grok Bot VM.

There are edge cases (for example, if you instruct the Bot very carefully to stream the local file without saving a full copy), but the normal, reliable workflow for a local video is to copy it over first.

you asked

message time: 2026-08-16 09:18:43
So are there ever times when I might request a huge amount of Grok model resources that either would be rate limited or face additional charges?

grok response

Yes. Heavy or continuous use of Grok Bot can hit both soft limits (the included weekly allowance) and, in some cases, trigger additional charges.

How usage works with Grok Bot

- Grok Bot access is included with **Cursor Ultra**, **Cursor Teams Premium seats**, or **SuperGrok Heavy**.

- Paid access includes a **weekly usage allowance** (the exact size is not publicly published in tokens or dollars).
- Usage is compute-metered and drawn from that shared weekly pool. Complex multi-step agent tasks, long-running routines, or many concurrent Bots consume the pool faster than ordinary chat.

What happens when you use a lot

1. **Within the weekly allowance**
   Normal operation. No extra charge.
2. **You exhaust the weekly allowance**
   Further Grok Bot work pauses or switches to **on-demand / extra usage** pricing (billed according to the actual model and token cost).
   You can usually buy Extra Usage Credits or enable auto top-up.
3. **Rate limits**
   There are also underlying request/token-per-minute style limits (similar to the xAI API tiers). Extremely bursty or high-volume use can temporarily hit these, though for most individual users the weekly pool is the more common constraint.
4. **No hard Grok Bot-specific spend cap (yet)**
   Official docs state there is currently **no Grok Bot-specific spend cap**. Account-level on-demand controls still apply, and usage/spend is visible on the Cursor dashboard.

Practical summary

- Everyday or moderate use stays inside the included weekly pool.
- Running many Bots nonstop, heavy transcription + analysis pipelines, long research routines, or continuous automation can burn through the allowance and move you onto on-demand billing.
- You can monitor live usage and the weekly reset in the relevant Settings / Usage dashboard.

Would you like the current best places to check your exact remaining allowance and any on-demand rates?

Powered by AI Exporter